

IoT Project Oulu: User Guide

Project description:

In this project you will build a low-power IoT setup where temperature, humidity, and soil moisture are being measured through sensors (Adafruit SHT40 and Sparkfun soil moisture sensor) communicating with a central microcontroller (STM32 Nucleo L412KB). The data gathered from the sensors will then be forwarded to your smartphone through a dedicated BLE module (Seeed Studio XIAO-nRF52840). The system will be ambiently powered using a solar panel, a supercapacitor and a Power Management Unit (BQ25570EVM). Throughout the project you will not only learn to make the setup work, but also to perform power consumption measurements using a PPK2 power profiler kit and optimizing the setup to consume as little as possible.

Comprehensive Project Build Documentation: STM32 to BLE Telemetry

This document outlines the full system architecture, hardware wiring, peripheral configuration, and data handling logic required to replicate your STM32-to-BLE telemetry node.

I. System Overview

The architecture is divided into two primary domains:

- 1. **Sensing Domain (STM32 Nucleo L412KB):** Responsible for I2C communication (SHT40), ADC sampling (Soil Moisture), and payload assembly.
- 2. **Communication Domain (Seeed Xiao nRF52840):** Functions as a transparent UART-to-BLE bridge, packetizing incoming serial data for wireless transmission.

II. Hardware Interfacing & Wiring

Connect the hardware (using a breadboard and jumper cables) according to the following table (Connection pin → Nucleo pin) and ensure all connections are verified with a multimeter before applying power.

Component	Connection Pin	STM32 Pin	Nucleo pin	Logic
SHT40	SDA	PB7	D4	I2C1_SDA
SHT40	SCL	PB6	D5	I2C1_SCL
SHT40	Vin	VDD	3v3	3.3V power
Moisture	VCC	PB1	D6	GPIO Output (High=On, Low=Off)
Moisture	Data	PA4	A3	ADC1_IN9
Xiao BLE	TX	PA10	D0/RX	RX (UART1)
Xiao BLE	RX	PA9	D1/TX	TX (UART1)

Component	Connection Pin	STM32 Pin	Nucleo pin	Logic
Xiao BLE (Only when not powered by USB-C)	3v3	VDD	3v3	3.3V power
All	GND	GND	GND	Shared Reference
Computer	USB port	PA2 (TX) and PA3(RX)	USB port	TX(UART2) and RX(UART2)

Critical Warning: When powering the boards independently via USB, **do not connect the 3.3V pins between the Nucleo and the Xiao**, as this may cause voltage regulator contention. You **must** connect the GND pins to establish a common voltage reference.

III. STM32 Development (CubeMX & CubeIDE)

1. CubeMX Configuration

- **Start a new project in CubeMX:** Use the board selector and choose NUCLEO-L412KB
- **Clock Configuration:** Go to 'Clock Configuration' and set MSI RC to 4 MHz
- **USART1 (Bridge):** in 'Pinout & Configuration' under 'Connectivity' make sure USART1 is set to Asynchronous mode, Baud rate: 115200, Word Length: 8 bits, Parity: None, Stop Bits: 1.
- **USART2 (Console):** in 'Pinout & Configuration' under 'Connectivity' make sure USART2 is set to Asynchronous mode, Baud rate: 115200 (for PC debugging via printf).
- **I2C1 (SHT40):** in 'Pinout & Configuration' under 'Connectivity' make sure I2C1 is set to Standard mode (100kHz), Addressing mode: 7-bit.
- **ADC1 (Moisture):** in 'Pinout & Configuration' under 'Analog' make sure ADC1 is set to Channel 9: Single-ended, Resolution: 12-bit.
- **Pinout Config:** In the 'Pinout View' set all the pins to the correct configuration according to the hardware interface table above (map relevant STM32 pin to logic).
- **Toolchain/IDE setting:** In 'Project Manager' choose a name for your project and set Toolchain/IDE to 'STM32CubeIDE'
- **Generate code:** Save the project and press 'GENERATE CODE' to generate a .ioc file.

2. CubeIDE Project

- **Import CubeMX project:** Click 'Create/Import STM32 project', then choose 'import STM32 project → STM32CubeMX/STM32CubeIDE project' and choose your CubeMX project file.
- **Setup done:** Now you have setup your project within STM32CubeMX. All the necessary configurations are already done there because of the code generation in CubeMX. When you now change your pinout configuration or any setting within your CubeMX project, this will also be synced in your CubeIDE project.
- **Implement user code:** Now you can use the space within '/* USER CODE BEGIN PV */' and '/* USER CODE END PV */' for declaring the private variables you need for your sensor and communication implementation code. You can use the space within '/* USER CODE BEGIN 0 */' and '/* USER CODE END 0 */' for making a Read_Sensors(void) and Transmit_Data(void) function, where you write the code to extract data from the sensors through I2C and ADC, and to transmit data to the BLE

module over UART. Finally, you can use the space within `/* USER CODE BEGIN WHILE */` and `/* USER CODE END WHILE */` to write the main program which calls the `Read_Sensors()` and `Transmit_Data()` functions. *For more information on the `Read_Sensors()` and `Transmit_Data()` code, see section V and VI*

IV. BLE Firmware

1. **Arduino IDE setup:** Open the Arduino IDE and fill the sketch with code to make the BLE module establish connection with a device in range (your smartphone), read out the serial UART data from the STM32, and send the sensor data to the smartphone over BLE.
2. **Flash BLE module:** Now, compile the sketch (verify) and connect your BLE module to your computer via USB-C (to be safe, first remove the BLE module from the breadboard). Now under 'tools → port' look for the serial port with the BLE module. If you cannot find it, double press the reset button on the BLE module (with something pointy e.g., a pen) and check again. This should make the port visible. Then, select the port and flash the device (upload).

V. Sensor interfacing basics:

Before writing your `Read_Sensors()` function, keep these hardware characteristics in mind:

- **SHT40 (Temperature & Humidity):** This digital sensor communicates over the I2C protocol. You will need to send a specific command byte to trigger a high-precision measurement, wait for the conversion, and then read back the data bytes.
- **SparkFun Soil Moisture Sensor:** This is an analog sensor read via the microcontroller's ADC. To prevent rapid corrosion of the physical sensor probes in the soil (electrolysis), do not power this sensor continuously. You must power it dynamically by toggling the connected GPIO pin HIGH just before taking your ADC reading, and immediately toggling it LOW once the reading is complete.

VI. Data Transmission Protocol: Packet Framing

Why Framing is Necessary: Standard UART serial communication is a continuous stream of bytes, whereas BLE transmits data in discrete chunks (MTUs). If the STM32 sends a long string of sensor data, the BLE module might chop the string in half to fit it into a radio transmission. This results in the smartphone receiving partial, broken data (e.g., receiving 45.0,324 instead of 31.8,45.0,324).

To prevent this, we use Start/End Framing. This creates a protective "envelope" around the data so the BLE module knows exactly when a message begins and ends.

The Frame Structure:

We wrap the sensor data in non-printable ASCII control characters:

- STX (Start of Text): 0x02
- ETX (End of Text): 0x03

The final transmitted packet looks like this: [0x02] 31.8,45.0,324 [0x03]

Implementation Strategy:

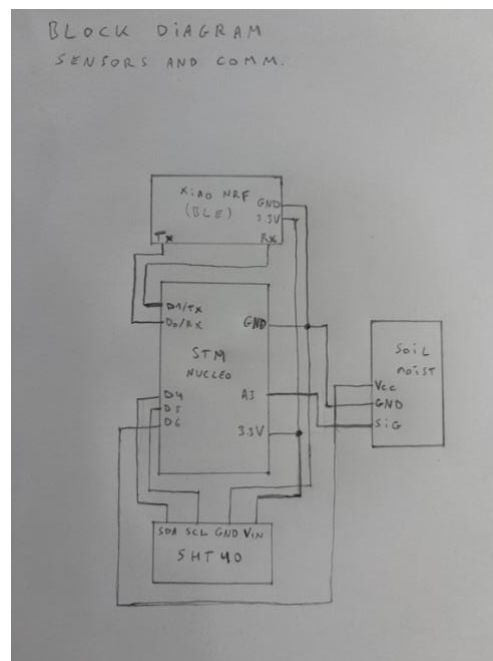
1. On the STM32 (Transmitter): Instead of just sending the string, you will transmit the 0x02 byte, then your data string, and finally the 0x03 byte using the `HAL_UART_Transmit()` function.
2. On the BLE Module (Receiver): In your Arduino code, you will create a state machine. The Xiao will ignore all incoming serial data until it sees the 0x02 byte. It will then save

the data into a buffer until it sees the 0x03 byte, at which point it will transmit the complete, unbroken string to the smartphone.

VII. Final setup and testing

1. **Flashing the STM32:** Within STM32CubeIDE, once you have your final code, build/compile the code using the hammer icon. If it builds successfully, connect your STM32 module to your computer using a micro USB cable and press the green 'play button' (triangle) to upload/flash your code to the device.
2. **Testing the system:** Download the 'nRF Connect for Mobile' app (by Nordic Semiconductor) on your smartphone. Using the app, connect to your BLE module, subscribe to the service (downward arrow), and watch your data coming in.

The block diagram for the full setup can be found here:



VIII. Debugging options

If you are having trouble with receiving the data on your smartphone, try the following things for troubleshooting:

1. **Serial monitoring:** Using the Arduino IDE serial monitor (or another serial monitoring tool like CoolTerm), you can look at the sensor data that you are sending to the BLE module if you add some extra serial transmissions in your STM code. This way you can find out where the problem is located (e.g., if you can see the correct sensor output values in the serial monitor, but not in the BLE connection, then you know that the problem is not the sensors, but rather the communication between your STM and the BLE module or the BLE firmware).
2. **Double check wiring:** Double check that all the wiring is correct according to the table above. Also double check if the table is correct by taking a look at the specific datasheets and user guides for all the components.
3. **Double check CubeMX configuration:** Double check that all the configurations outlined in this guide are reflected in the CubeMX configuration.

4. **Baud rate mismatch:** Double check that the baud-rate for the UART connections are the same on both sides of the communication. If the STM32 is configured to use 115200, then this should also be used in the BLE module and in the serial monitor on your computer.
5. **Common ground:** Always make sure that all the components (STM32, BLE module, SHT40, and soil moisture sensor) share a common ground.
6. **Further trouble shooting:** For further troubleshooting, using detailed prompts about your specific problem in an LLM can be very useful.

IX. External power supply

Now that we have the whole system running, it is time to make it work with external 3.3V power supply rather than USB power. For this we need to make some minor hardware adaptations to the STM32 board (see STM32 Nucleo-32 Boards User Manual

https://www.st.com/resource/en/user_manual/um1956-stm32-nucleo32-boards-mb1180-stmicroelectronics.pdf).

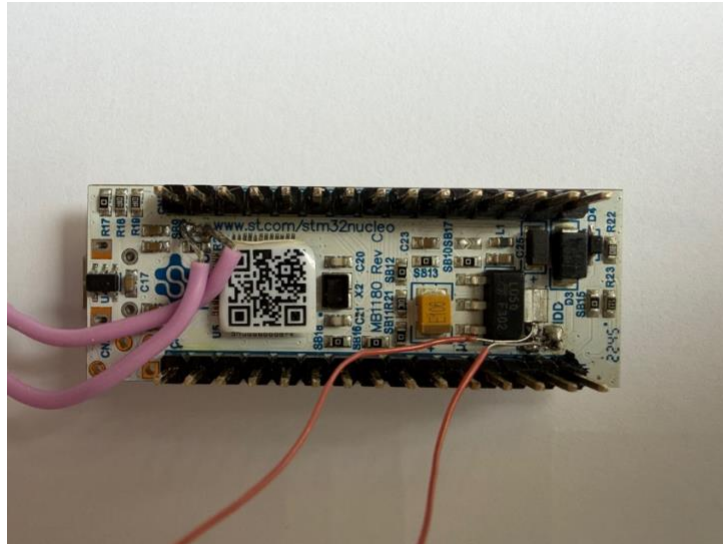
We want to work with a 3.3V power supply as this allows us to disable the ST-LINK programmer component on the board which has a high power consumption and is not needed for regular operation of our system (only for flashing the device).

1. **Desolder SB14:** This bridge forms the connection between the boards 3.3V power and the ST-LINK programmer. By desoldering this bridge, we disable the high-consuming ST-LINK part. Solder some thin flexible wires to both ends of the bridge and add some pin headers to the ends of the wires such that we can still close the bridge whenever we want to flash/reprogram the device.
2. **Desolder SB9:** This bridge connects the ST-LINK reset line to the MCU. By desoldering this bridge, we avoid the STM32 going into a permanent reset state. Also in this case, solder some thin flexible wires to both ends of the bridge and add some pin headers to the ends of the wires such that we can still close the bridge whenever we want to flash/reprogram the device.

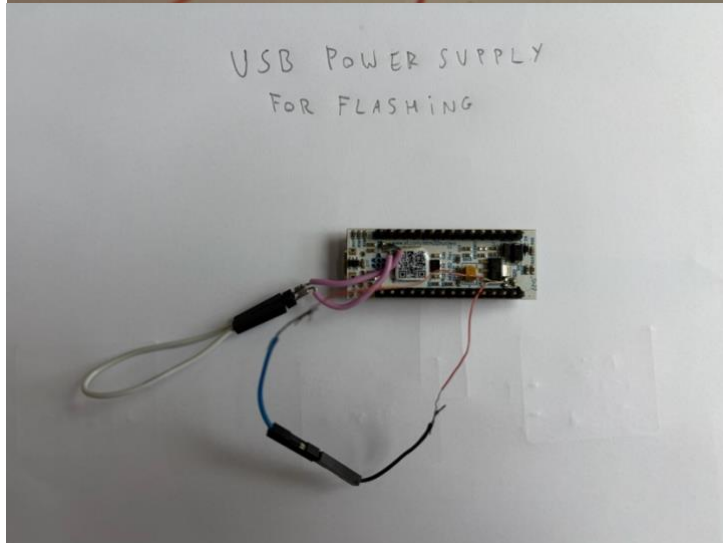
Now it is **very important to always keep this in mind:**

- When powering the system with external 3.3V power supply: **Leave SB14 and SB9 open**
- When powering the system with USB power: **Close SB14 and SB9**
- **Never power the device through USB while external power is also supplied.**
- If the system is not functioning as it should, use a multimeter to double check all of the connections, especially the small (delicate) wires to SB14 and SB9, and make sure that your power supply + is connected to the STM32 board 3v3 pin and the power supply – is connected to the STM32 board GND.

The adapted STM Nucleo board with the wires on SB14 and SB9 should look something like this:



USB POWER SUPPLY
FOR FLASHING



EXTERNAL 3.3V
POWER SUPPLY



X. Power supply via PMU

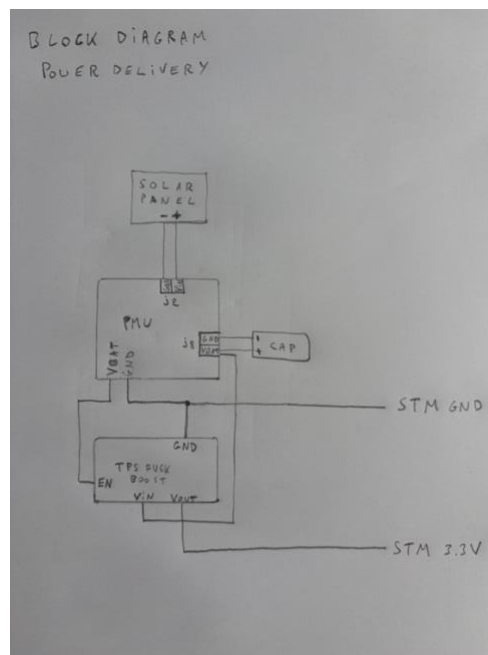
Now that we have tested the whole system with an external power supply set to 3.3V, it is time to hook up our Power Management Board (PMU) and see if we can run the system using a supercapacitor.

1. **Preparing the buck boost convertor:** First off we will solder pin headers on the through hole connections of the TPS63020 buck boost convertor board. Solder on a pin at GND, VIN, VOUT, and EN. Once this is done, make sure the right output voltage is set. Looking at the board, you will see 4 solder bridges in a vertical row: T1, T2, T3, and T4. These are used to configure the output voltage of the board to 2.5, 3.3, 4.2, or 5 V respectively. (see instruction manual: <https://manuals.plus/asin/b0d28pwqm8?nonce=691159>). Desolder any bridge that does not correspond to 3.3V and make sure the 3.3V bridge (T2) is soldered (closed).
2. **Testing the buck boost convertor:** Now validate the performance of the buck boost convertor using the power supply. First connect the Enable pin to the VIN pin to set it to be always enabled when there is power input. Then set your power supply to 5V and connect the power supply – to GND on the board and the power supply + to VIN. Now check with the multimeter if you read out 3.3V on VOUT. Now change the input voltage from 5 to 2V and check if you still read out 3.3V on the output from the board. If this is both the case, the convertor works as it should.
3. **Using the supercapacitor:** Check the voltage on the supercapacitor and make sure it is above 1.8V. If it is lower, charge it to 4V using the power supply as follows: Put the output voltage of the power supply to **maximum 4V and set the current limit to 1A max**. Then connect the positive end (typically long end) of the supercap to the + of the power supply and the negative end to the – of the power supply. Now validate with a multimeter that the capacitor voltage is slowly rising. Once it is charged up to the desired 4V, disconnect it from the power supply. Now that we have a supercap with a voltage between 1.8V and 5.5V, we can connect the **positive end to VIN and the negative end to GND on the buck boost board**. Now also connect **GND on the buck boost board to GND on your STM board**. Finally, connect **VOUT to the 3.3V pin on the STM** and validate if the program is running as it should (i.e., you are receiving your BLE packets with the correct data).
4. **Using the PMU:** Since we will be using a PMU that allows for max 4.2V from the capacitor, **make sure it is not charged to more than 4.1V**. If your capacitor is already pre-charged to more than 4.2V, make sure to drain it to a slightly lower level by running the STM-board with the power coming straight from the buck boost convertor using the setup from the previous step (4). Once this is done, **connect the capacitor with the negative end to the GND terminal on the Power Management Unit (PMU) and the positive end to the VBAT terminal**. Then **connect the VIN from the buck boost board to the VBAT pin on the PMU, the GND pin from the buck boost board to the GND pin on the PMU and of the STM, and the EN pin of the buck boost board to the BAT_OK pin on the PMU**. Now validate if it works correctly by connecting the **VOUT from the buck boost board to the 3.3V pin of the STM**. The BAT_OK pin serves as protection mechanism. When the cap voltage gets low (around 2.8V), the BAT_OK pin will no longer receive power, and therefore, the buck boost board will be disabled and the whole application will be shut down. As soon as the supercap hits around 3V again, the BAT_OK pin will be activated again and the application will be powered again.
5. **Adding solar charging:** Finally, to make our setup completely self-sustained, we will connect a solar panel to the PMU to slowly charge the supercapacitor with solar energy. First, we need to make sure the **PMU board configuration** is correct. Set the headers for JP4, JP2, and JP3 as follows:
 1. **JP4 (80% to VOC_SAMP):** MPPT is active.
 2. **JP2 (/EN to GND):** The main energy harvester is turned ON.
 3. **JP3 (VOUT_EN to VSTOR):** The internal buck converter is ON (though you aren't using its output, leaving it on only costs about 400nA, which is harmless)

Then connect the negative end of the solar panel to GND and the positive end to VIN (J2). Now, the supercap will be receiving energy from the solar panel and you should be able to see it slowly charge when it is disconnected from the load. To test if everything works as it should. Perform the following procedure:

1. Charge the supercap to about 3.2V
2. Connect the load (application) and let the supercap drain to under 2.8V
 1. You should now see the safety mechanism come into action, automatically disconnecting the load
3. Let the capacitor slowly charge using the solar panel (place it at the window with a lot of sunlight or alternatively use the flashlight from your phone (slower)).
 1. Once the supercap reaches around 3V, the load should automatically connect again, and you will see the application starting again.
4. *Debug tip: If you have your supercap connected to the PMU with a voltage higher than 3.1V and you see that the BAT_OK pin does not get triggered, shortly connect VBAT to VSTOR with a jumper cable. After this you should see the BAT_OK pin working again as it should.*

The block diagram for the final harvesting setup can be found here:



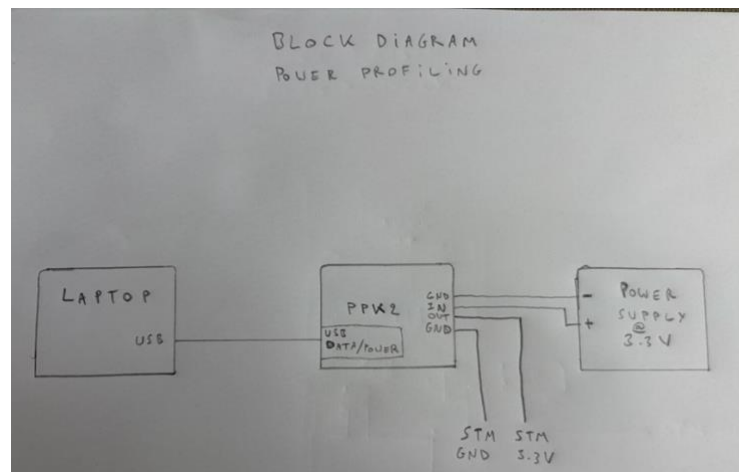
XI. Power profiling

Now that we have the full setup ready, we can dive into the power profiling and optimization. To make things easier, we will disconnect the buck boost and PMU setup from our system and **power everything with the power supply set to a voltage of 3.3V and current limit of 100mA.** To measure how much energy our system consumes, we can use the nRF Power Profiling Kit (PPK2). Set it up as follows:

1. **Install the software:** If not yet installed, download and install nRF Connect for Desktop on your computer and open the app 'Power Profiler'.
2. **Connect your PPK2 board:** Plug the PPK2 board into your laptop using a micro-USB cable to the Data/Power port and turn it on using the onboard switch. Then click 'select device' in the app and connect to your board

3. **Wiring:** We want to wire the PPK2 board in series in between the power source and the STM. Use the 4-pin connector and connect as follows:
 1. **VIN to + of the power supply**
 2. **VOUT to 3.3V pin of the STM**
 3. **Share the GND between the power supply, the PPK2, and the STM.**
4. **Sampling:** Set the power supply mode in the app to 'Ampere Meter' and click 'start' to start sampling. You will now see the power profile of the whole system being measured. After hitting stop we can choose to save the logged profile as a .ppk file or a .csv file for further analysis.
5. **Basic analysis:** You will first see a large spike in current draw at the start of the process when the system is booting. Then the profile will stabilize and you will clearly see a periodic pattern with a 'floor consumption' of around 10mA and then larger spikes of around 25-30mA caused by the BLE communication. This is still a completely unoptimized profile. We will look into lowering the floor and minimizing the communication overhead in the last part of this project.

The block diagram for the profiling setup can be found here:



XII. Power optimizations

Designing a sensor node to operate perpetually on harvested ambient energy (e.g., solar via a BQ25570 PMU and supercapacitor) requires a fundamental shift in firmware architecture. "Out-of-the-box" microcontroller and BLE settings are optimized for rapid development and responsiveness, keeping peripheral clocks active and consuming continuous milliamperes (mA) of current.

To achieve a viable batteryless system, the architecture must be heavily optimized to drop the baseline current into the microampere (μA) range. This section details the theoretical and practical modifications made to the system's duty cycle, MCU power states, and BLE transmission architecture to achieve this.

1. Duty Cycle Extension: Relaxing the Measurement Interval

The most direct method to reduce average power consumption is extending the time the system spends asleep. The original firmware executed a measurement and transmission cycle every 5 seconds.

- **The Power Impact:** With a 5-second interval, the active time (waking up, reading I2C/ADC sensors, and transmitting) constitutes a significant percentage of the system's total runtime. This aggressive duty cycle does not allow a small ambient energy

harvester sufficient time to replenish the supercapacitor, leading to rapid voltage collapse.

- **The Optimization:** The sleep interval can be extended to minutes or even hours. By strictly limiting active windows to once every x minutes/hours, the average power consumption plummets, allowing the PMU to maintain a net-positive energy balance in the supercapacitor during optimal harvesting conditions.

2. MCU Deep Sleep: Transitioning from Active Wait to STOP2 Mode

To really make the optimization in the previous step useful, we need to make sure that our 'sleep' is actually low power. In standard prototyping, pausing execution is typically handled by `HAL_Delay()`.

- **The Active Wait Penalty:** `HAL_Delay()` is an "active wait." The CPU is still fully powered, executing NOP (No Operation) instructions, and the high-speed system clocks remain active to allow the SysTick timer to fire every millisecond. This keeps the MCU drawing several milliamperes continuously while "doing nothing."
- **The STOP2 Architecture:** To eliminate this, `HAL_Delay()` can be completely removed in favor of the STM32's **STOP2** low-power mode. When entering STOP2, the CPU completely halts execution, the main high-speed clocks (MSI/PLL) are shut down, and most peripherals are powered off. Only the Low-Speed External (LSE) oscillator remains active to drive the Real-Time Clock (RTC), which serves as the alarm to wake the system x minutes later. This drops the MCU's baseline consumption into the microwatt range.

To make this work, make sure to first make the following configurations in CubeMX:

To implement the 30-second STOP2 cycle, the RTC must be configured to generate an external interrupt (EXTI) that forces the CPU to wake up. Configure the following settings in the .ioc Device Configuration Tool:

1. **Activate the RTC:** In the left pane, expand **Timers** and select **RTC**. Check the box for **Activate Clock Source** and check the box for **Internal WakeUp**.
2. **Route the Low-Power Clock:** Open the **Clock Configuration** tab. Locate the RTC Clock multiplexer and ensure it is routed to the **LSE** (Low-Speed External) oscillator, which consumes significantly less power than the high-speed clocks.
3. **Set the 30-Second Alarm:** Return to the RTC menu and open the **Parameter Settings** tab.
 - Set the *Wake Up Clock* to **1Hz**. This scales the clock to a 1 Hz (1 tick-per-second) frequency.
 - Set the *Wake Up Counter* to **30**.
4. **Enable the Wake-Up Interrupt (Crucial):** Open the **NVIC Settings** tab within the RTC menu. Check the box to enable the **RTC wake-up interrupt through EXTI line**. *Note: If this interrupt is not enabled, the RTC will count to 30, but the CPU will remain permanently asleep.*

3. Mitigating Silicon-Level Leakage

Simply commanding the MCU to sleep does not guarantee a microamp floor due to hardware-level parasitic drains:

- **Floating Pins:** Any unpowered, disconnected, or floating GPIO pin acts as an antenna. It will oscillate between logic states, causing the input buffers to continuously switch and leak hundreds of microamps.
- **The GPIO fix:** To solve this, an initialization should be executed right before entering STOP2 mode. Every single pin on Port A and Port B should be forced into an Analog High-Impedance state. This physically disables the digital input/output buffers, blocking all leakage paths to the peripherals (such as the I2C sensors and BLE module). When the

RTC wakes the MCU x minutes later, the standard `MX_Init` functions should be called to dynamically restore only the necessary pins for the active measurement window

- **LED losses:** Any LEDs that are still active during operation of our system should be disabled as they consume unnecessary power. For the SHT40, this means physically desoldering the LED from the breakout board.

4. BLE Architecture Shift: From Connected to Burst Beaconing

The most significant power consumption in any IoT node is the wireless transmission. The original architecture we deployed used a "Connected" BLE UART profile, which is highly inefficient for sporadic sensor data.

- **The Connected Overhead:** In a connected state, the BLE peripheral (the Seeed Xiao) and the central device (the gateway/smartphone) must remain synchronized. Even if no sensor data is being transmitted, the BLE module is forced to wake up every few milliseconds (the Connection Interval) to transmit a "heartbeat" packet to prevent the connection from dropping. This forces the module's High-Frequency Clock (HFCLK) to remain permanently active, raising the power floor.
- **Connectionless Broadcast (Beacon) Mode:** The optimized architecture completely abandons the connected state. Instead, the STM32 passes the sensor data to the BLE module, which parses the floats into raw hexadecimal bytes and injects them directly into its public Advertising Packet (Manufacturer Specific Data). The gateway (phone) simply scans the airwaves and reads the data passively without ever initiating a power-hungry handshake.
- **The 2-Second Burst:** To push power savings to the absolute physical limit, the BLE module can operate in a "Burst" configuration. Upon receiving data from the STM32, the Xiao can broadcast its beacon aggressively (every 100ms) to ensure the gateway catches it, but *only for exactly 2 seconds*. After 2 seconds, the FreeRTOS idle task should manually terminate the radio and suspends the CPU, resulting in zero radio emissions and true microamp silence for the remaining time of the sleep cycle.

5. BLE Hardware-Level Optimizations

To support the Burst Beacon architecture, several hardware-level optimizations should be made to the nRF52840 BLE module:

- **Hardware Interrupt Wake-Up:** Instead of keeping the BLE UART active to listen for the STM32 (which requires the HFCLK), the BLE module should sleep entirely. The STM32 should send a single "Dummy Byte" (0xFF) to trigger a physical hardware interrupt on the BLE module, waking it up just in time to receive the actual payload.
- **Transmission Power Reduction:** Wireless transmission is the most energy-intensive operation in the system. The original code utilized a transmit (TX) power of +4 dBm, prioritizing maximum physical range but creating high peak-current spikes. The TX power can be reduced to -4 dBm. This trades unnecessary long-distance range for a significantly lower peak current during the 2-second broadcast window.
- **DC-DC Converter Activation:** The nRF52840's internal, highly efficient buck converter should be explicitly enabled in software, dropping the active radio transmission current by up to 30% compared to the default LDO regulator.
- **Killing LEDs:** All onboard RGB LEDs, as well as the hidden library-controlled status LEDs should be explicitly driven HIGH (off) in the setup routine to prevent baseline static current draw.

Summary of Power Profile Impact

By combining extended duty cycling, strict MCU STOP2 deep sleep, hardware-level pin isolation, LED detachment, and the Burst Beacon broadcast architecture, the system's power

profile will be fundamentally transformed. Instead of a continuous multi-milliamp (~10mA) baseline punctuated by high-current radio heartbeats (~25mA), the optimized node exhibits a much lower static floor (~0.6mA), interrupted only by brief, highly efficient limited active windows with much lower peaks (~12mA) every 30 seconds. This profile is much more suited for sustained, autonomous operation on standard energy-harvesting hardware.

Power profile before optimization with full BLE connection and sensing+communication every 5 seconds:

10 second window:

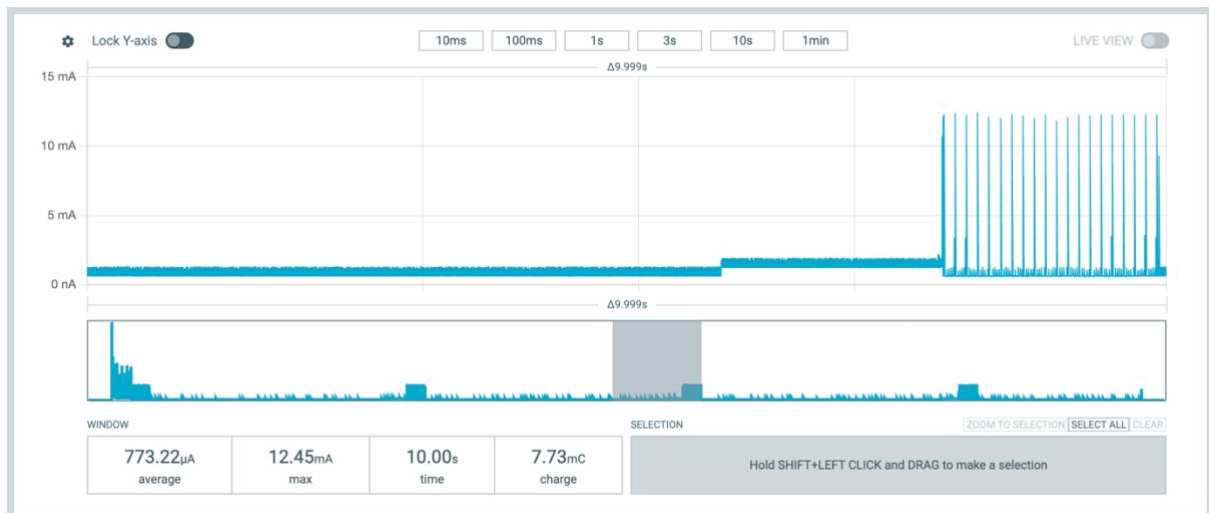


1 second window:



Power profile after optimization with BLE burst beaconing and sensing+communication every 30 seconds:

10 second window:



1 second window in STOP2:



1 second window in active sensing:



1 second in BLE burst beaconing:

